Архитектура СУБД PostgreSQL

Экземпляры сервера
Кластер баз данных
Схемы и объекты в PostgreSQL
Системный каталог

Малков О.Б., 2023

Экземпляр сервера PostgreSQL

Малков О.Б., 2023

Экземпляр сервера PostgreSQL (1)

- PostgreSQL – программа, относящаяся к классу систем управления базами данных. Когда она выполняется, ее называют **сервером PostgreSQL** или **экземпляром сервера**.
- **Экземпляр (instance) сервера** – СУБД, находящаяся в фазе выполнения, связанная с конкретной БД и готовая выполнять запросы на обработку этой БД. На одном сервере может быть несколько экземпляров.
- Экземпляр **не использует** совместно системные и пользовательские БД с другими экземплярами на том же компьютере. Но экземпляры совместно используют **физические ресурсы сервера** (центральный процессор, ОЗУи диски).

Экземпляр сервера PostgreSQL (2)

- **Текущий экземпляр сервера** – тот сервер БД, с к-рым выполнено соединение в одной из программ (утилита командной строки *psql* или графическая утилита *pgAdmin*).
- Каждый экземпляр **полностью независим** от других с точки зрения безопасности, данных, которыми управляет, и в других отношениях.
- На логическом уровне у двух экземпляров на одном компьютере столько же общего, сколько у двух экземпляров на разных компьютерах.

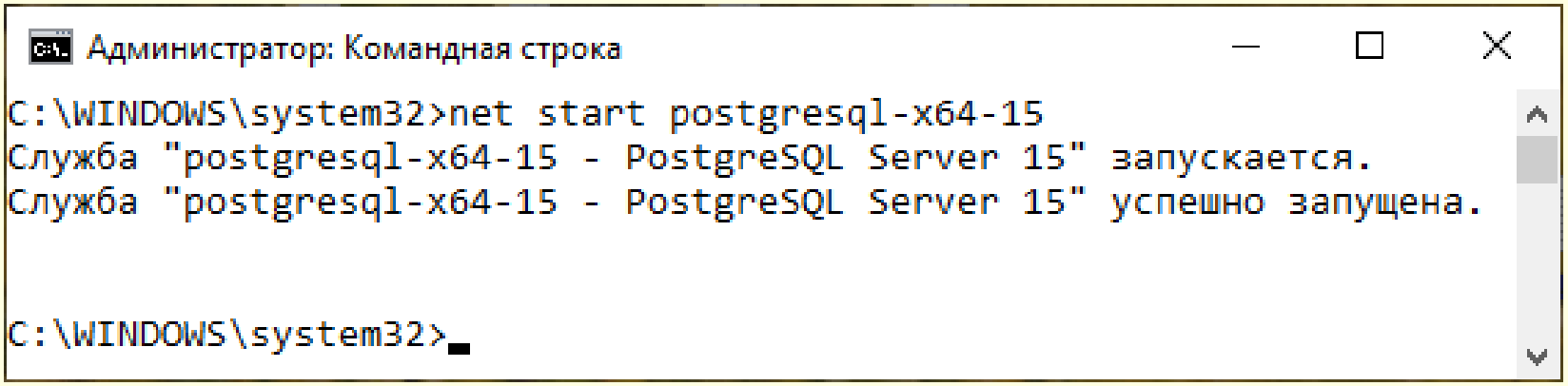
Управление экземпляром сервера

- К основным операциям управления сервером относятся:
  - начальная инициализация кластера баз данных;
  - запуск сервера;
  - останов сервера;
  - обновление параметров конфигурации и др.
- Запустить, остановить или перезапустить сервер PostgreSQL в ОС Windows можно различными способами:
  - из командной строки (или в программе PowerShell);
  - с использованием утилиты pg_ctl;
  - с использованием диспетчера служб ОС Windows.

Запуск сервера с помощью команды *net start*

- Запустите командную строку Windows от имени администратора и выполните команду ***net start***, чтобы запустить сервер PostgreSQL:

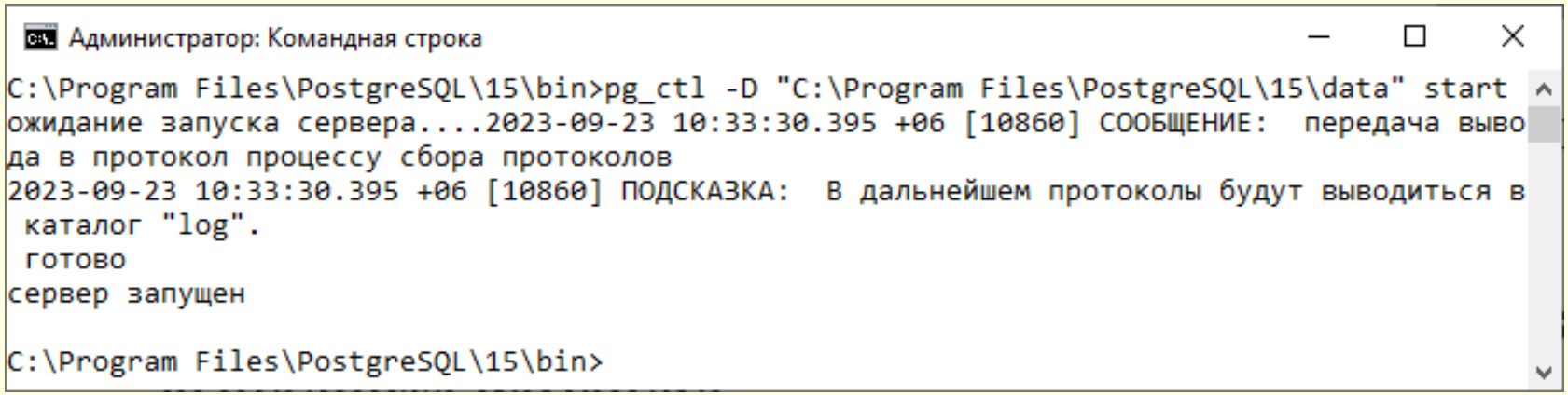
C:\WINDOWS\system32>net start postgresql-x64-15



Запуск сервера с помощью утилиты *pg_ctl*

- Перейдите в каталог установки PostgreSQL (по умолчанию – *C:\Program Files\PostgreSQL\15\bin*).
- В командной строке Windows от имени администратора выполните команду:

pg_ctl -D "C:\Program Files\PostgreSQL\15\data" start



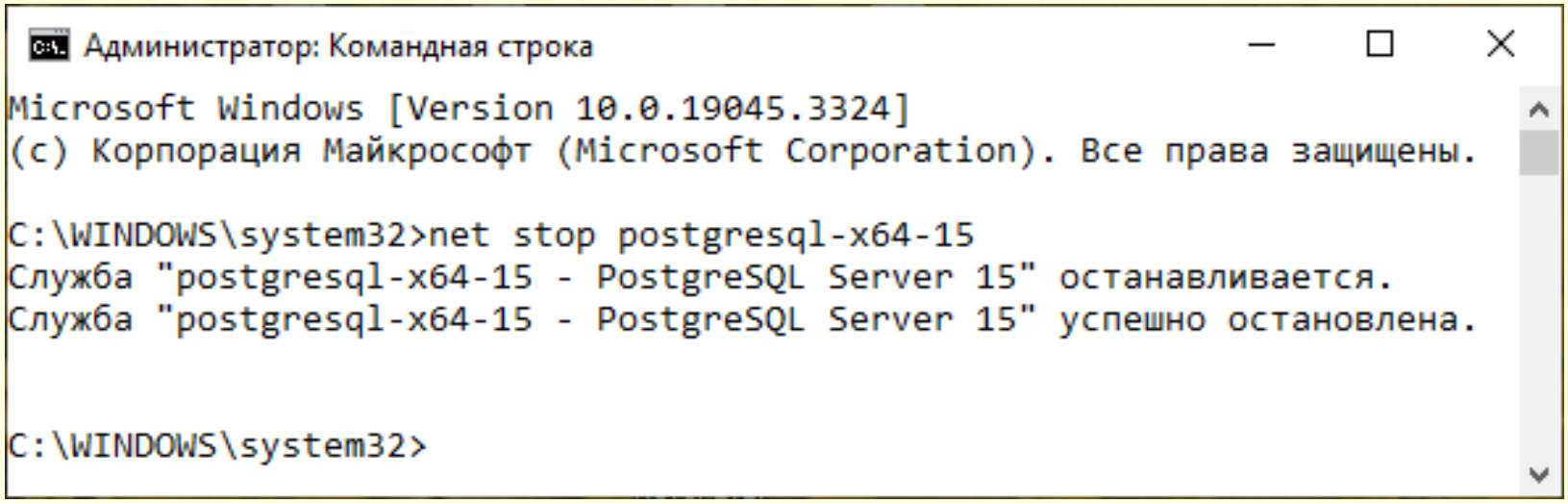
Запуск сервера с помощью диспетчера служб

- Нажмите *Win* + *R*, чтобы запустить окно *Выполнить*.
- Введите *services.msc* и нажмите *ОК*.
- В диспетчере служб выберите службу *postgresql-x64-15* и нажмите *Запустить*.
- После этого статус службы изменится на *Выполняется*.

Останов сервера с помощью команды *net stop*

- Запустите командную строку Windows от имени администратора и выполните команду ***net stop***, чтобы остановить сервер PostgreSQL:

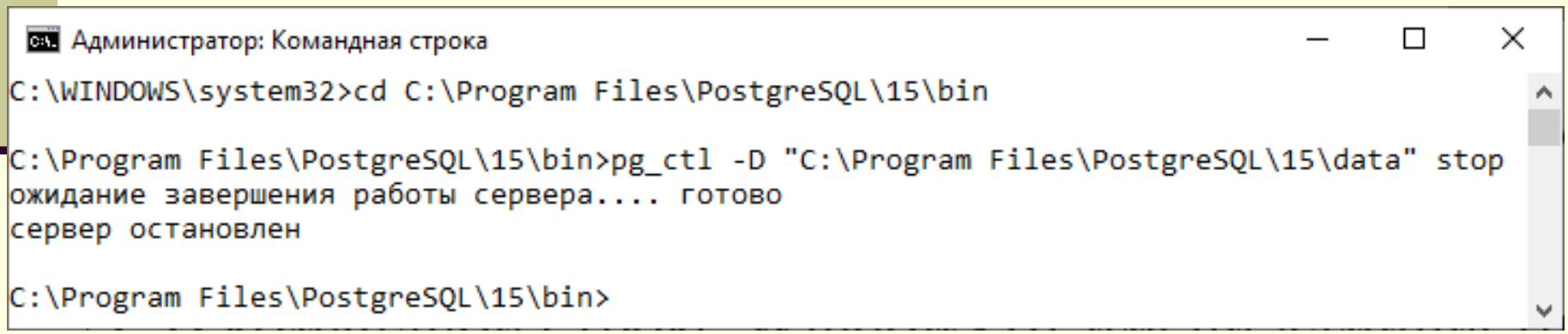
C:\WINDOWS\system32>net stop postgresql-x64-15



Останов сервера с помощью утилиты *pg_ctl*

- Перейдите в каталог установки PostgreSQL (по умолчанию – *C:\Program Files\PostgreSQL\15\bin*).
- В командной строке Windows от имени администратора выполните команду:

pg_ctl -D "C:\Program Files\PostgreSQL\15\data" stop



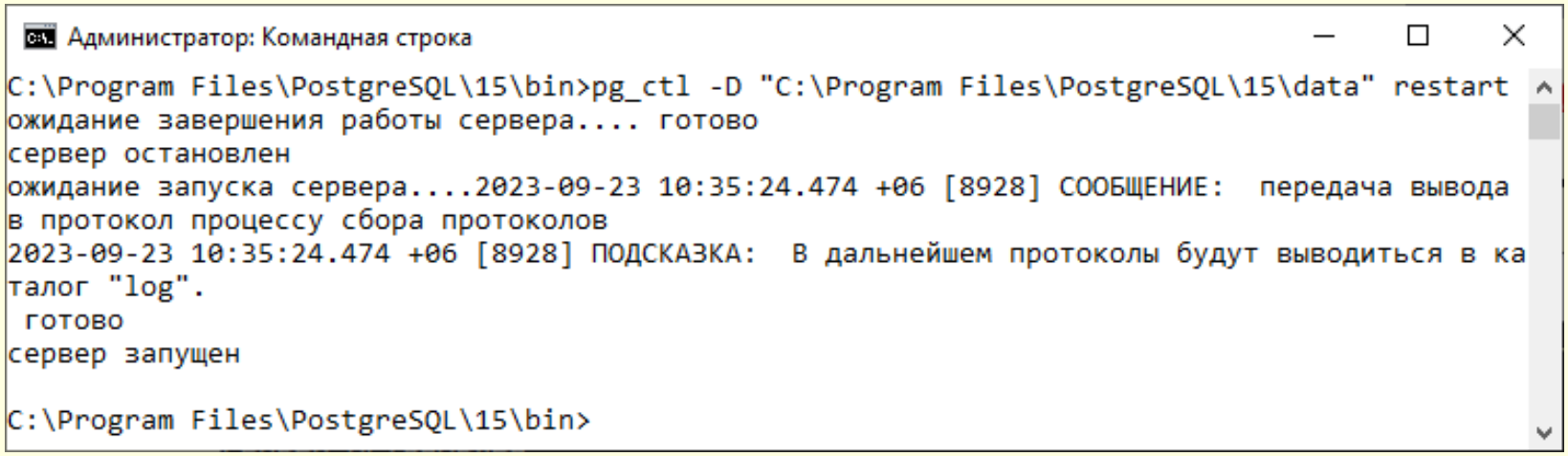
Останов (приостанов) сервера с помощью диспетчера служб

- Нажмите *Win* + *R*, чтобы запустить окно *Выполнить*.
- Введите *services.msc* и нажмите *ОК*.
- В диспетчере служб выберите службу *postgresql-x64-15* и нажмите *Остановить*.
- После этого статус службы изменится на *Выполняется*.
- Чтобы приостановить сервер, нажмите *Приостановить* в диспетчере служб или выполните команду *net pause postgresql-x64-15* из командной строки.

Два способа перезапуска сервера

- Способ 1: перезапуск сервера с помощью ***утилиты pg_ctl***:

  pg_ctl -D "C:\Program Files\PostgreSQL\15\data" restart



- Способ 2: перезапуск сервера с помощью диспетчера служб. В диспетчере служб выберите службу *postgresql-x64-15* и нажмите *Перезапустить*.

Журнал сообщений сервера

Журнал сообщений сервера (1)

- Информация о ходе работы СУБД записывается в **журнал сообщений сервера**. Сюда записываются:
  - **служебные сообщения сервера** (сведения о запуске и останове сервера, служебные сообщения о проблемах);
  - **сообщения пользовательских сеансов** (сообщения о выполняющихся командах, времени их работы, о возникающих блокировках и т. п.)
  - **сообщения приложений** (разработчики приложений могут записывать в журнал свои собственные сообщения).
- Настройки PostgreSQL позволяют гибко определять, какие сообщения и в каком формате должны попадать в журнал. (вывод **в формате csv** удобен для автоматизации анализа).

Журнал сообщений сервера (2)

- При установке PostgreSQL на Windows, логи по умолчанию сохраняются в *C:\Program Files\PostgreSQL\15\data\pg_log*, где 15 – это конкретная версия установленной PostgreSQL.
- В этой папке вы найдете файлы журнала, соответствующие каждому запущенному экземпляру PostgreSQL.
- Файлы журнала имеют расширение *".log"* и названия, состоящие из даты и времени с момента запуска PostgreSQL.
- Журнал можно открыть любым текстовым редактором. Каждая запись в журнале начинается с даты и может состоять из нескольких строк. Записи за сегодня будут в конце файла.

Настройка сервера PostgreSQL

Малков О.Б., 2023

Параметры конфигурации (1)

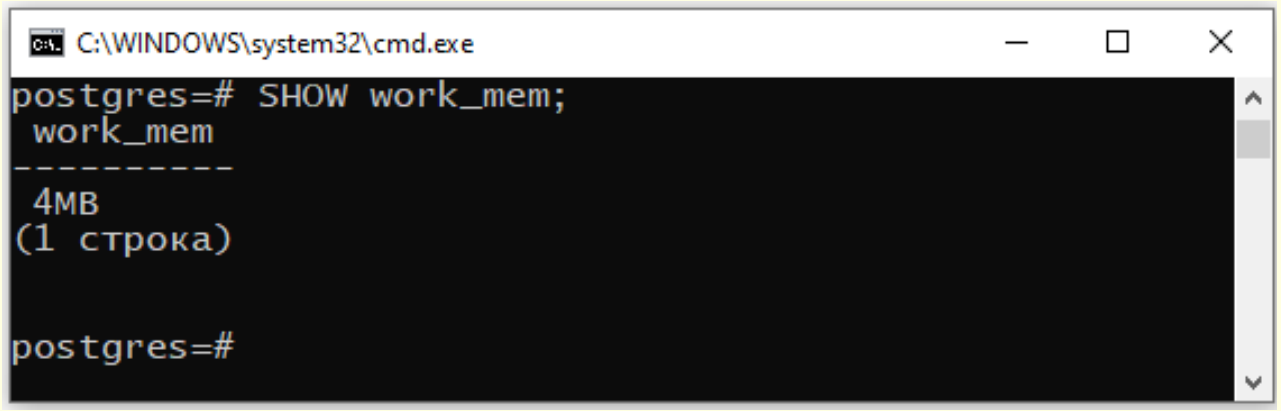
- Настройка сервера PostgreSQL:
  - управление потреблением ресурсов;
  - настройка служебных процессов и пользоват. сеансов;
  - управление журналом сервера и многих др. задач.
- Сервер настраивают с помощью **параметров конфигурации**. Конфигурационные файлы, определяющие настройки сервера, располагаются в каталоге баз данных:
  - *C:\Program Files\PostgreSQL\15\data\postgresql.conf* – осн. конфиг. файл, содержащий значения параметров сервера;
  - *C:\Program Files\PostgreSQL\15\data\pg_hba.conf* – файл, определяющий настройки доступа и другие файлы.

Параметры конфигурации (2)

- Важно знать, как проверить текущие значения параметров конфигурации и установить новые.
- Основной файл *postgresql.conf* редактируется вручную. Второй конфигурационный файл – *postgresql.auto.conf* – изменяется специальной командой *ALTER SYSTEM*.
- Параметры, установленные через *ALTER SYSTEM*, имеют приоритет над параметрами в *postgresql.conf*.
- Большинство параметров конфигурации допускает изменение значений в пользовательских сеансах прямо во время выполнения. Текущие значения доступны в представлении *pg_settings*.

Пример – работа с параметром *work_mem* (1)

- Параметр *work_mem* задает объем памяти, к-рый будет использоваться для внутр. операций сортировки и хеш-таблиц, прежде чем будут задействованы временные файлы на диске:



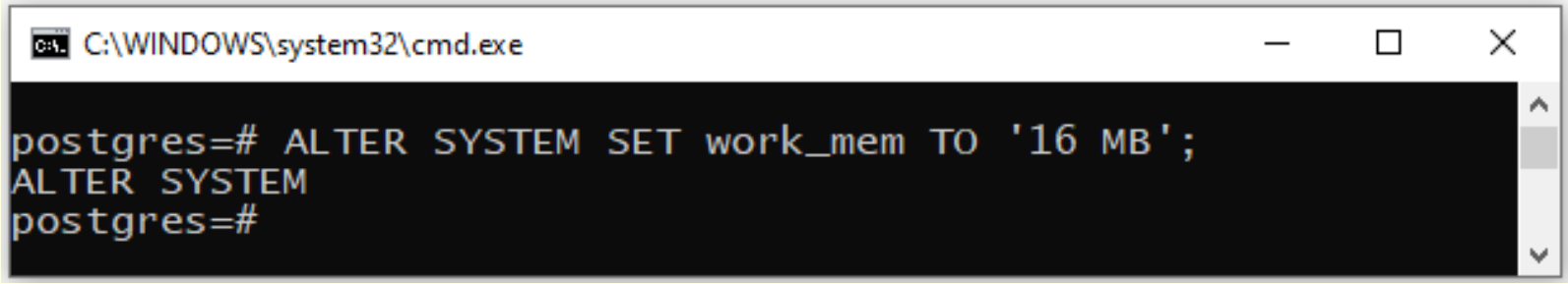
- 4MB – значение по умолчанию, оно слишком мало. Мы хотим увеличить его до 16MB для всего экземпляра.
- Есть два пути.

Пример – работа с параметром *work_mem* (2)

- Внести изменение в *postgresql.conf*, раскомментировав и изменив строку, где определяется параметр:

#work_mem = 4MB # min 64kB

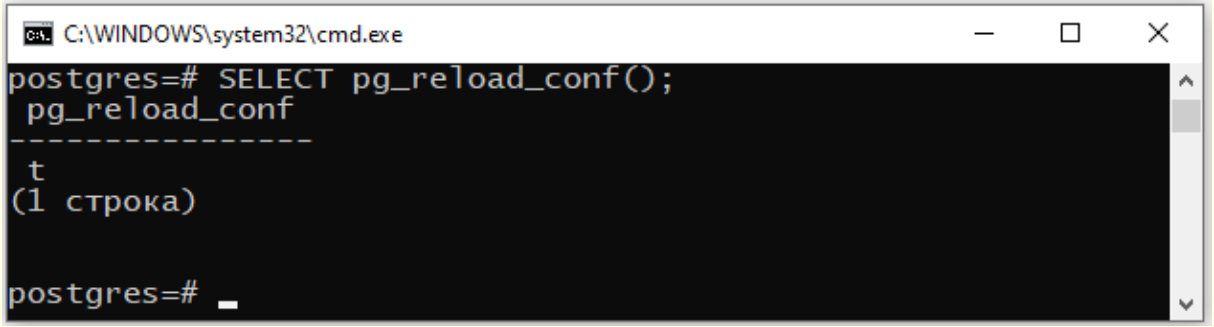
- Или можно изменить параметр с помощью команды SQL:



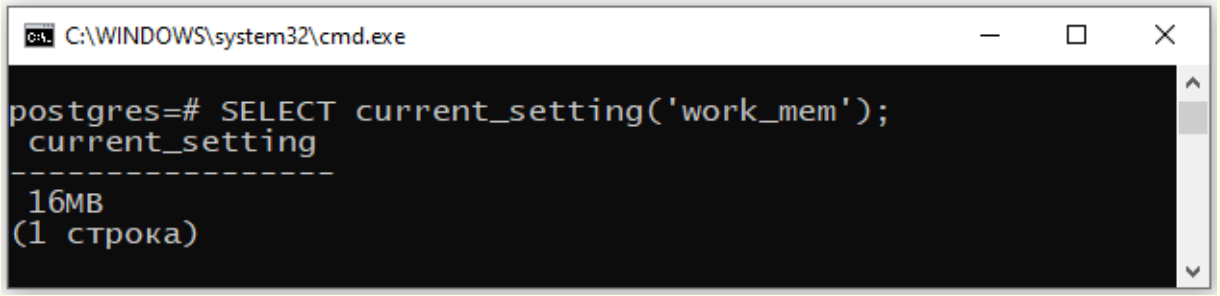
- Такое изменение попадает не в *postgresql.conf*, а в другой файл – *postgresql.auto.conf*.

Пример – работа с параметром *work_mem* (3)

- Чтобы изменение вступило в силу, нужно перечитать конфигурационные файлы. Можно использовать функцию SQL:

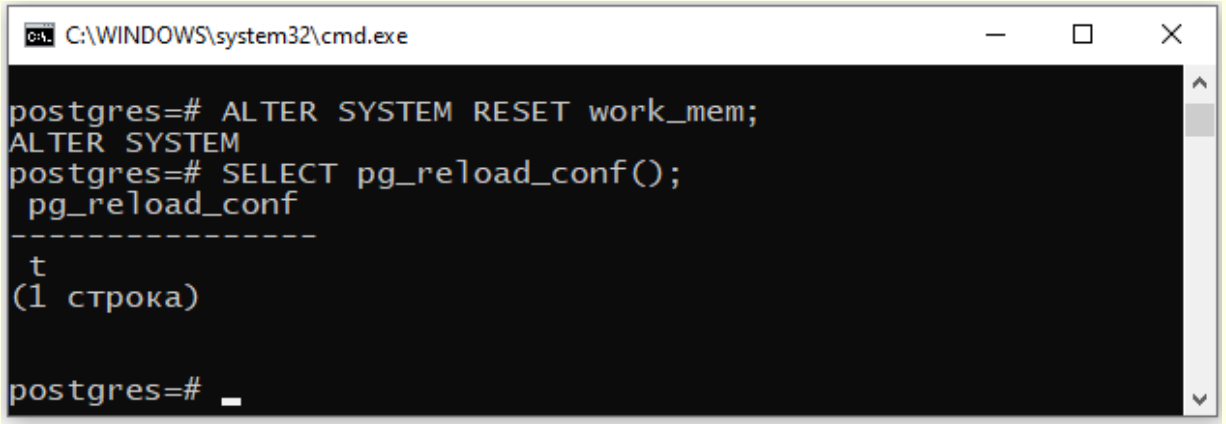


- Проверим, что новое значение параметра применилось. Кроме команды *SHOW* можно сделать это следующим образом:



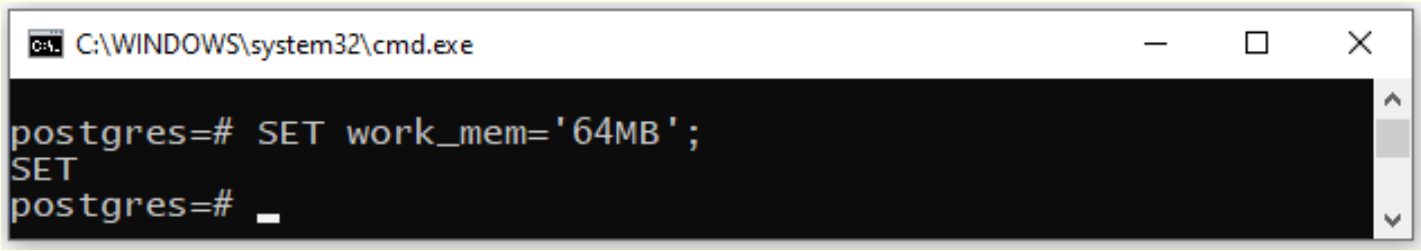
Пример – работа с параметром *work_mem* (4)

- Чтобы восстановить значение параметра по умолчанию, достаточно вместо *SET* использовать *RESET* (и перечитать конфигурационные файлы):

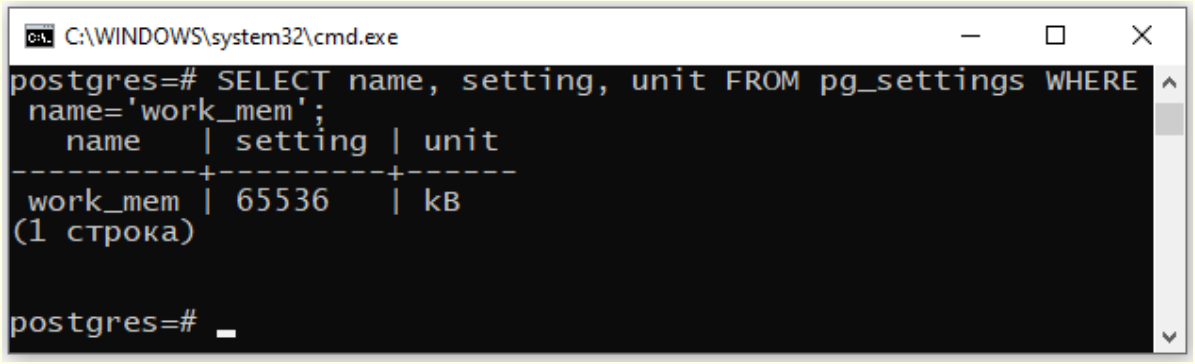


- Большинству параметров можно установить новое значение для текущего сеанса во время выполнения. Если мы хотим выполнить запрос, сортирующий большой объем данных, то для сеанса можно увеличить значение *work_mem*:

Пример – работа с параметром *work_mem* (5)



- Проверим, что значение применилось, еще одним способом – выполним запрос **к представлению**:



- Новое значение действует только **в текущем сеансе** или **в текущей транзакции** (при указании *SET LOCAL*).

Параметры конфигурации (итог)

- Для всего экземпляра:
  - основной файл параметров – *postgresql.conf* ;
  - *ALTER SYSTEM – postgresql.auto.conf* .
- Для текущего сеанса:
  - *SET / RESET;*
  - *set_config* .
- Просмотр текущих значений:
  - *SHOW ;*
  - *current_setting( );*
  - *pg_settings.*

Типовой сценарий изменения параметров для всей системы

1. Вносим изменения в файл *postgresql.conf* или выполняем команду

   ALTER SYSTEM SET parameter TO new_value;

2. Обновляем конфигурацию (вызываем функцию *pg_reload_conf*). Для некоторых параметров потребуется перезагрузка сервера.

3. Проверяем, что изменения применились. Представление *pg_settings* содержит актуальные значения всех параметров.

Кластер баз данных

Данные, которыми управляет PostgreSQL, хранятся в **базах данных**.

Один экземпляр PostgreSQL одновременно работает с несколькими базами данных. Этот набор баз данных называется **кластером баз данных**.

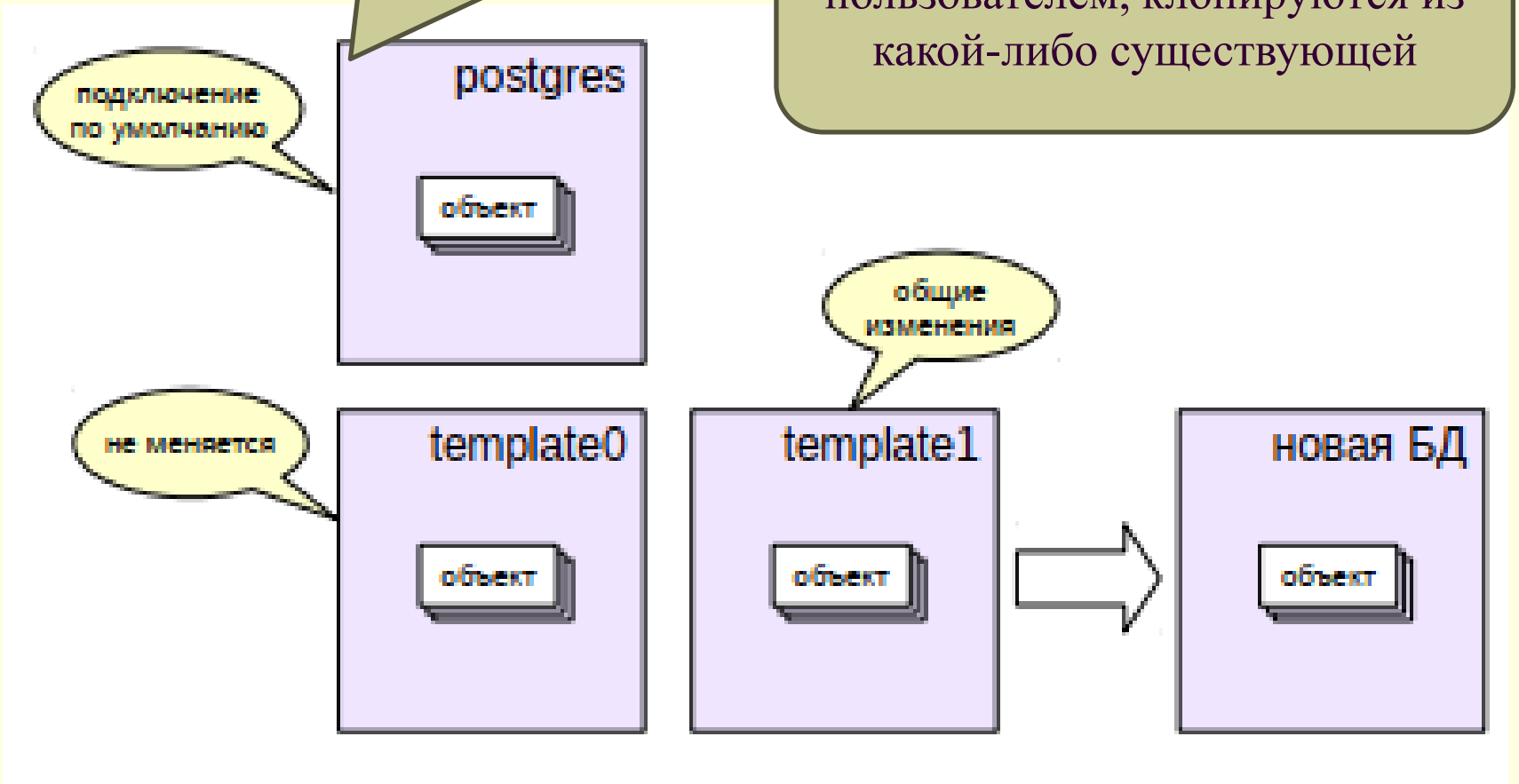
Кластер баз данных (2)

- При инициализации кластера (автоматически при установке PostgreSQL либо вручную командой *initdb*) в *PGDATA* создаются **три одинаковые базы данных**:
- ***template0*** – используется, например, для восстановления из логической резервной копии или для создания базы в другой кодировке и никогда не должна меняться;
- ***template1*** – служит шаблоном для всех БД, к-рые может создать пользователь в этом кластере (добавляют объекты и расширения, к-рые будут копироваться в каждую новую БД);
- ***postgres*** – представляет собой обычную базу данных, к-рую можно использовать по своему усмотрению (не удалять!).

Кластер баз данных (3)

Не является обязательной, но некоторые утилиты предполагают ее наличие

Все остальные БД, создаваемые пользователем, клонируются из какой-либо существующей



Список баз данных

```
postgres=# \l
                                  Список баз данных
   Имя    | Владелец | Кодировка |     LC_COLLATE      |
----------+----------+-----------+---------------------+
 postgres | postgres | UTF8      | Russian_Russia.1251 |
 proba    | postgres | UTF8      | Russian_Russia.1251 |
 sales    | postgres | UTF8      | Russian_Russia.1251 |
 template0| postgres | UTF8      | Russian_Russia.1251 |
          |          |           |                     |
 template1| postgres | UTF8      | Russian_Russia.1251 |
          |          |           |                     |
 testDB   | postgres | UTF8      | Russian_Russia.1251 |
(6 строк)
```

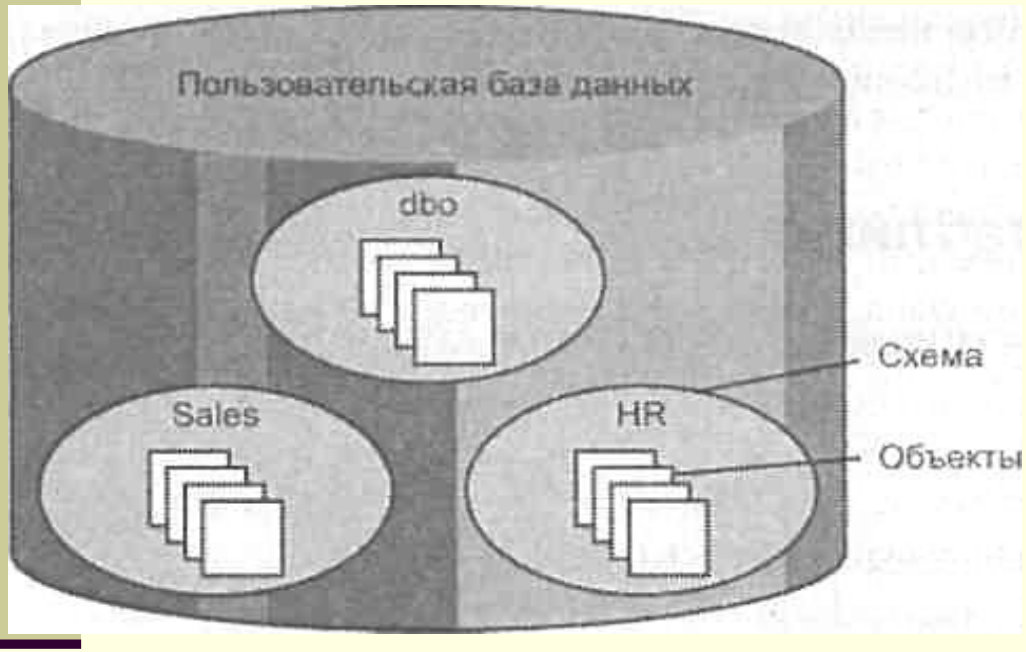
```
postgres=# CREATE DATABASE data_logical;
CREATE DATABASE
postgres=# \c data_logical
Вы подключены к базе данных "data_logical" как пользователь "postgres".

data_logical=#
```

Схемы и объекты в PostgreSQL

Малков О.Б., 2023

Схемы и объекты (1)



- БД включает в себя схемы (schema), а схемы содержат объекты.
- Именно схемы служат контейнерами объектов (таблиц, представлений, хранимых процедур и др.).

- Схемы принадлежат базам данных, все объекты БД распределены по каким-либо схемам.

Схемы и объекты (2)

- Схему можно рассматривать как **пространство имен** (name-space) для всех объектов в базе данных. Имена объектов одного типа в схеме должны быть **уникальными**.
- Схемы позволяют разделить объекты на **логические группы** для управления ими, предотвратить конфликты имен при работе нескольких пользователей или приложений.
- В PostgreSQL схема и пользователь – разные сущности (хотя настройки по умолчанию упрощают работу с одноименными схемами).
- Кроме пользовательских схем, имеется несколько специальных **служебных**.

Специальные схемы (обычно есть в каждой БД)

- ***public*** – используется по умолчанию для пользовательских объектов, если не выполнены иные настройки.
- ***pg_catalog*** – хранит объекты системного каталога. Альтернативное представление системного каталога (определенное в стандарте SQL) дает схема ***information_schema***.
- ***pg_temp*** – служит для хранения временных таблиц. На самом деле таблицы создаются в схемах *pg_temp_1*, *pg_temp_2* и т. п. – у каждого пользователя своя схема. Но обращаются все пользователи к ней как к ***pg_temp***.
- Есть и другие схемы, но они носят технический характер.

Зачем объединять объекты в схемы?

- Повышается **уровень безопасности данных**. На уровне схемы можно управлять правами доступа, предоставляя права на использование объектов только указанным группам пользователей – **принципалам**.
- Схема позволяет выполнить «упаковку» групп объектов, что облегчает манипулирование привилегиями.
- Будучи **пространством имен**, схема используется как префикс в имени объекта. Например, в схеме с именем *Sales* (Продажи) есть таблица *Orders* (Заказы). Составное имя объекта – *Sales.Orders*.

Работа со схемами (просмотр)

- Для списка схем в psql есть специальная команда *\dn*:

```
data_logical=# \dn
        Список схем
  Имя   |      Владелец
--------+--------------------
 public | pg_database_owner
(1 строка)
```

- Эта команда не показывает служебные схемы. Чтобы увидеть их, нужно добавить модификатор *S*:

```
data_logical=# \dnS
              Список схем
        Имя           |      Владелец
----------------------+--------------------
 information_schema   | postgres
 pg_catalog           | postgres
 pg_toast             | postgres
 public               | pg_database_owner
(4 строки)
```

Работа со схемами (создание)

```
data_logical=# CREATE SCHEMA special;
CREATE SCHEMA
data_logical=# \dn
        Список схем
   Имя   |      Владелец
---------+--------------------
 public  | pg_database_owner
 special | postgres
(2 строки)
```

создание новой схемы

```
data_logical=# CREATE TABLE tbl(n integer);
CREATE TABLE
data_logical=#
```

создание таблицы (схема public)

```
data_logical=# \dt public.*
         Список отношений
 Схема  | Имя |   Тип   | Владелец
--------+-----+---------+----------
 public | tbl | таблица | postgres
(1 строка)
```

список таблиц в схеме public

Работа со схемами (перемещение объектов)

```
data_logical=# ALTER TABLE tbl SET SCHEMA special;
ALTER TABLE
data_logical=#
```

перемещение в схему special

```
data_logical=# \dt public.*
Отношение "public.*" не найдено.
data_logical=#
```

что осталось в схеме public?

```
data_logical=# \dt special.*
          Список отношений
  Схема  | Имя |   Тип   | Владелец
---------+-----+---------+----------
 special | tbl | таблица | postgres
(1 строка)
```

а в схеме special?

Работа со схемами (составные имена объектов)

```
data_logical=# SELECT * FROM special.tbl;
 n
---
(0 строк)
```

таблица переместилась, к ней можно обращаться с явным указанием схемы

```
data_logical=# SELECT * FROM tbl;
ОШИБКА:  отношение "tbl" не существует
СТРОКА 1: SELECT * FROM tbl;
                        ^
```

если опустить имя схемы, таблица не будет найдена

Составные имена объектов

- Если при обращении к объекту схема не указана явно, выбирается первая подходящая схема из перечисленных в пути поиска.
- Путь формируется на основе значения параметра *search_path*, к которому неявно добавляются схемы *pg_catalog* и (при необходимости) *pg_temp*. Это позволяет иметь в разных схемах объекты с одинаковыми именами.
- Рекомендуется всегда использовать составные имена (разрешение имени объекта требует расходов; если в разных схемах есть объекты с одним именем, можно получить не тот объект).

Путь поиска (1)

- При указании объекта надо определить, о какой схеме речь, т.к. в разных схемах могут храниться объекты с одинаковыми именами:
  - если имя объекта квалифицировано именем схемы, то используется явно указанная схема;
  - если имя без квалификатора, то проверяются схемы, перечисленные в пути поиска, к-рый определяется конфигурационным параметром *search_path*.
- Путь поиска может отличаться от значения *search_path*.
- Можно провести аналогию между путем поиска *search_path* и путем *PATH* в операционных системах.

Путь поиска (2)

- В путь поиска не включаются несуществующие схемы из *search_path*, а также схемы, к которым у пользователя нет доступа.
- В начало пути поиска неявно добавляются:
  - схема *pg_catalog*, чтобы всегда иметь доступ к системному каталогу;
  - схема *pg_temp*, если пользователь создавал временные объекты.
- При создании нового объекта с именем без квалификатора он попадает в первую явно указанную в пути схему (не считая несуществующих и тех, на которые нет прав).

Путь поиска (2)

- Путь поиска по умолчанию имеет следующее значение:

```
data_logical=# SHOW search_path;
   search_path
-----------------
 "$user", public
(1 строка)
```

- Конструкция "$user" обозначает схему с тем же именем, что и имя текущего пользователя (в нашем случае – *student*). Поскольку такой схемы нет, она игнорируется.

- Чтобы не думать над тем, какие схемы есть, каких нет, и какие не указаны явно, можно воспользоваться специальной функцией.

Путь поиска (3)

- Реальный путь поиска (включая неявные схемы) возвращает вызов функции: *current_schemas(true)*.
- Схемы перебираются в указанном в пути поиска порядке, слева направо. Если в схеме нет объекта с нужным именем, поиск продолжается в следующей схеме.

```
data_logical=# SELECT current_schemas(true);
   current_schemas
---------------------
 {pg_catalog,public}
(1 строка)
```

- Видим, что была убрана несуществующая схема и неявно была добавлена схема системного каталога.

Путь поиска (4)

- Установим путь поиска, например, так:

```
data_logical=# SET search_path = public, special;
SET
data_logical=# _
```

- Теперь таблица будет найдена:

```
data_logical=# SELECT * FROM tbl;
 n
---
(0 строк)
```

- Конфигурационный параметр установлен на уровне сеанса (при переподключении значение пропадет). Устанавливать такое значение на уровне всего кластера тоже неправильно – возможно, этот путь нужен не всегда и не всем.

Путь поиска (4)

- Но параметр можно установить и на уровне отдельной базы данных:

```
data_logical=# ALTER DATABASE data_logical SET search_path = public, special;
ALTER DATABASE
data_logical=#
```

- Теперь он будет устанавливаться для всех новых подключений к БД *data_logical*. Проверим:

```
data_logical=# \c data_logical
Вы подключены к базе данных "data_logical" как пользователь "postgres".
data_logical=# SHOW search_path;
   search_path
-----------------
 public, special
(1 строка)
```

Системный каталог PostgreSQL

Малков О.Б., 2023

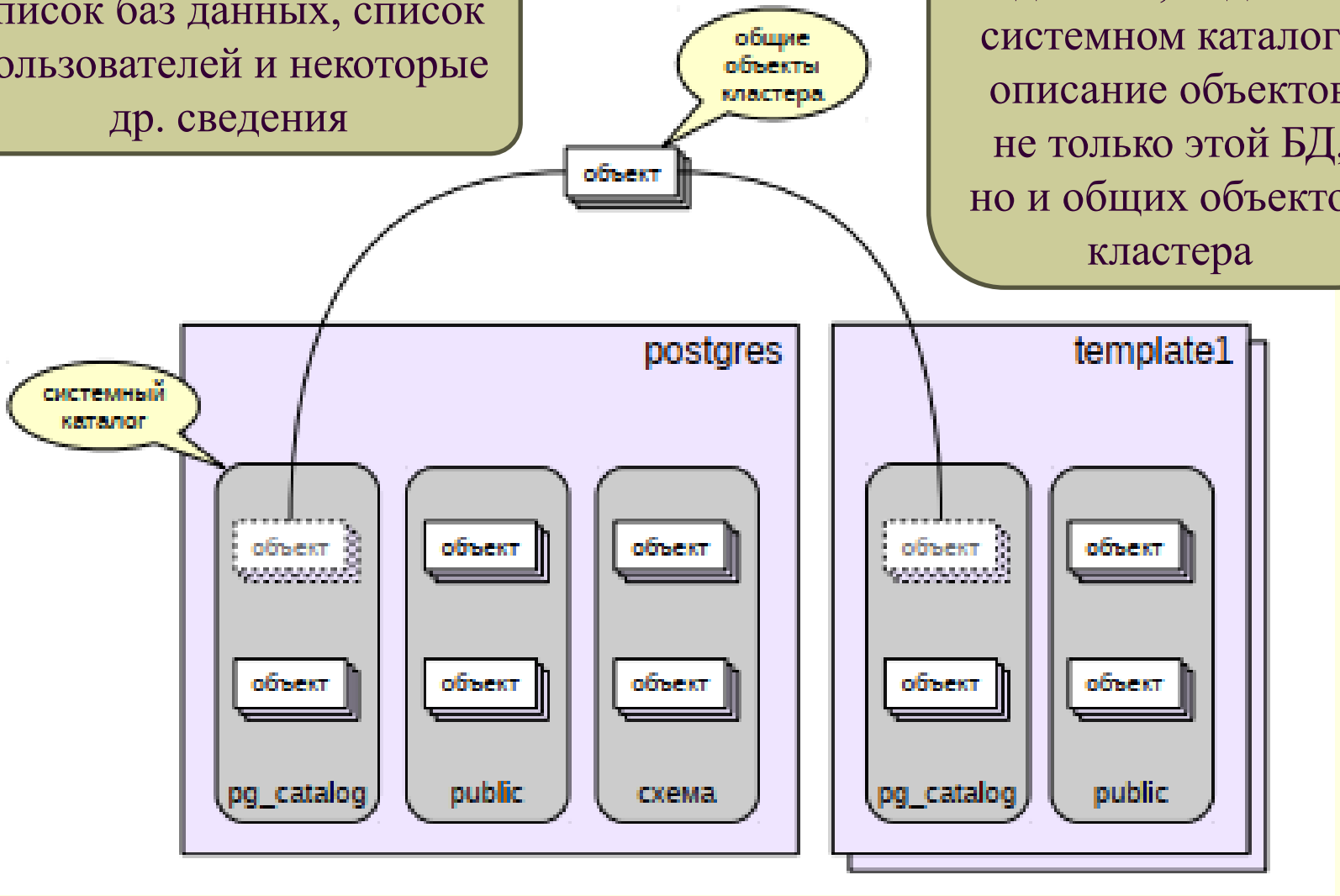
Системный каталог (1)

- Хранит **метаинформацию** обо всех объектах кластера. Включает набор **таблиц** в каждой БД (описывающих объекты этой БД) и несколько таблиц, общих для кластера и не принадлежащих какой-то отдельной БД.
- Для удобства над таблицами определены несколько **представлений**.
- Можно обращаться с помощью обычных запросов SQL, а psql имеет ряд команд для удобства просмотра.
- Таблицы системного каталога не обновляют напрямую. Они изменяются автоматически при выполнении **команд DDL**.

Базы и схемы кластера

список баз данных, список пользователей и некоторые др. сведения

Клиент, подключенный к БД данных, видит в системном каталоге описание объектов не только этой БД, но и общих объектов кластера



Системный каталог (2)

- Имена таблиц начинаются с *pg_* (например, *pg_database*). Столбцы таблиц начинаются с префикса, соответствующего имени таблицы (например, *datname*). Имена объектов хранятся в нижнем регистре (например, *'postgres'*).
- Таблицы системного каталога имеют **первичные ключи** и связаны **внешними ключами**.
- Первичные ключи не определены явно. В большинстве случаев в качестве первичного ключа выступает столбец *oid*, имеющий спец. тип *oid – object identifier* (целое 32-битное число).
- До версии PostgreSQL 12 этот столбец был скрытым (его можно было увидеть, только указав имя в списке *SELECT*).

Системный каталог (3)

- Чтобы вывести информацию о любых объектах, psql (и графические среды) обращается к таблицам системного каталога.

```
data_logical=# SELECT datname FROM pg_database;
   datname
-------------
 postgres
 testDB
 template1
 template0
 sales
 proba
 data_logical
(7 строк)
```

команда \l для получения списка баз данных кластера обращается к таблице

```
data_logical=# SELECT nspname FROM pg_namespace;
      nspname
--------------------
 pg_toast
 pg_catalog
 information_schema
 special
 public
(5 строк)
```

список схем находится к таблице

Системный каталог (4)

- Чтобы вывести информацию о любых объектах, psql (и графические среды) обращается к таблицам системного каталога.

```
data_logical=# SELECT relname, relkind, relnamespace
data_logical-# FROM pg_class WHERE relname = 'tbl';
 relname | relkind | relnamespace
---------+---------+--------------
 tbl     | r       |        41270
(1 строка)
```

тип *oid*

просмотр таких объектов, как таблицы и индексы

```
data_logical=# SELECT relname, relkind FROM pg_class
data_logical-# WHERE relnamespace = 'pg_catalog'::regnamespace LIMIT 5;
            relname              | relkind
---------------------------------+---------
 pg_statistic                    | r
 pg_type                         | r
 pg_foreign_table                | r
 pg_proc_oid_index               | i
 pg_proc_proname_args_nsp_index  | i
(5 строк)
```

список объектов в схеме pg_catalog

Удаление баз данных, схем и объектов (1)

- Можно ли удалить схему *special*?

```
data_logical=# DROP SCHEMA special;
ОШИБКА:  удалить объект схема special нельзя, так как от него зависят другие
ПОДРОБНОСТИ:  таблица tbl зависит от объекта схема special
ПОДСКАЗКА:  Для удаления зависимых объектов используйте DROP ... CASCADE.
data_logical=#
```

- Схему нельзя удалить, если в ней находятся какие-то объекты. Их надо перенести или удалить. Можно удалить схему вместе со всеми ее объектами:

```
data_logical=# DROP SCHEMA special CASCADE;
ЗАМЕЧАНИЕ:  удаление распространяется на объект таблица tbl
DROP SCHEMA
data_logical=#
```

Удаление баз данных, схем и объектов (2)

- Базу данных можно удалить, если к ней нет активных подключений:

```
data_logical=# \conninfo
Вы подключены к базе данных "data_logical" как пользователь "postgres"
 (сервер "localhost": адрес "::1", порт "5432").
data_logical=# \c postgres
Вы подключены к базе данных "postgres" как пользователь "postgres".
postgres=# DROP DATABASE data_logical;
DROP DATABASE
postgres=#
```